

Day 3 – Feature Engineering, Cross-Validation & Feature Selection

Task 1: Create & Justify Engineered Features

In Task 1, the Adult Income dataset was loaded and cleaned by replacing ? values with NaN. Eight new features were created to capture useful patterns that may not be represented directly by the original features.

Engineered Features

1. **age_bucket** – Groups age into meaningful career/age ranges.
2. **hours_bucket** – Groups weekly working hours into part-time, standard and overtime categories.
3. **capital_gain_flag** – Indicates whether a person has any capital gain.
4. **log_capital_gain** – Applies $\log(1 + \text{capital-gain})$ to reduce the effect of extreme values.
5. **higher_education** – Identifies individuals with higher education using education-num.
6. **education_hours_interaction** – Combines education level and working hours.
7. **capital_loss_flag** – Indicates whether a person has any capital loss.
8. **age_hours_interaction** – Combines age and weekly working hours.

Mutual Information was calculated for each engineered feature to measure its univariate predictive signal. A feature dictionary containing the feature name, type, creation rule, justification and predictive signal was also created.

Conclusion: Feature engineering introduced additional information that can help models capture income-related patterns more effectively.

Task 2: Rebuild Pipeline

In Task 2, the engineered features were integrated into a leakage-safe Scikit-learn pipeline using FunctionTransformer.

The pipeline followed this structure:

Raw Data → Feature Engineering → Imputation → Scaling/One-Hot Encoding → Model Preprocessing

- Numeric features: missing values were replaced using median imputation and then standardized using StandardScaler.
- Categorical features: missing values were replaced using the most frequent value and converted using OneHotEncoder(handle_unknown="ignore").

Logistic Regression and Random Forest pipelines were created using the same preprocessing.

Conclusion: Integrating feature engineering into the pipeline makes the complete process repeatable and prevents preprocessing leakage during cross-validation.

Task 3: Cross-Validated Model Comparison

Task 3 evaluated models using **5-Fold Stratified Cross-Validation** instead of relying on a single train/test split.

Three models were compared:

1. **Logistic Regression**
2. **Random Forest**
3. **Gradient Boosting**

The following metrics were measured for every fold:

- Accuracy
- F1 Score
- ROC AUC

For each model, the **Mean $\pm$ Standard Deviation** was reported. Fold-level results were also saved.

Boxplots were created for:

- F1 Score
- ROC AUC
- Accuracy

Conclusion: Cross-validation provides a more reliable estimate of model performance because the models are evaluated on multiple stratified folds. F1 Score was used as an important comparison metric because the Adult dataset has imbalanced income classes.

Task 4: Statistical Comparison & Feature Importance

In Task 4, the top two models were selected based on their mean F1 score from cross-validation.

A paired t-test and Wilcoxon signed-rank test were performed using the fold-level F1 scores to determine whether the performance difference between the two best models was statistically significant.

The following were also analyzed:

- Logistic Regression coefficients
- Random Forest feature importance
- Importance of engineered features
- Practical difference in F1 score

Feature-importance plots were created to identify the most influential features.

Conclusion: Statistical testing helped determine whether the difference between the top models was meaningful rather than simply caused by fold-to-fold variation. Feature importance also showed which original and engineered features contributed most to model predictions.

Task 5: Feature Selection & Dimensionality

Task 5 used SelectKBest with Mutual Information to reduce the number of input features.

Different values of K were tested and compared with the full-feature baseline.

For every configuration, the following were measured:

- Mean F1 Score
- F1 Standard Deviation
- ROC AUC
- Accuracy
- Cross-validation runtime

F1 performance and runtime were plotted against the number of selected features.

The best feature-selection configuration was automatically identified based on the observed mean F1 score.

Conclusion: Feature selection can reduce dimensionality and potentially improve efficiency while maintaining or improving predictive performance. The selected model and feature count can be used as a candidate configuration for tomorrow's hyperparameter tuning.

Overall Day 3 Conclusion

Day 3 focused on improving the Adult Income classification workflow through feature engineering, leakage-safe pipelines, cross-validation, statistical comparison and feature selection.

Eight meaningful engineered features were introduced and evaluated using Mutual Information. These features were integrated into repeatable Scikit-learn pipelines. Three classification models were compared using 5-fold Stratified Cross-Validation with Accuracy, F1 and ROC AUC.

The top models were statistically compared using paired tests, while feature importance analysis identified influential original and engineered features. Finally, SelectKBest with Mutual Information was used to evaluate dimensionality reduction and its effect on performance and runtime.

Overall, the experiments produced a more reliable and systematic model-selection process. The best-performing model and selected feature configuration can be carried forward to Day 4 for hyperparameter tuning and further optimization.